ResumeLens

Formal Language-Based Resume Screening

Explicit evidence ⟶ canonical skills ⟶ explainable patterns ⟶ validated profiles

Discrete Structures III | Integrative Task | October 2026

Víctor Team

01 Problem and objective

Résumés express equivalent skills using different names and place them in different orders. ResumeLens tests whether **explicitly reported qualifications** satisfy a defined professional pattern.

Research question

How can regular expressions, finite-state transducers, finite automata and a structured language form one traceable résumé-processing pipeline?

Four profiles, one solution

Full Stack Developer and Machine Learning Engineer are the reference profiles. Cloud & DevOps Engineer and Data Scientist are the team's additional profiles.

02 Shared architecture

1. Extract Python `re`
Source fragments and structured facts

↓

2. Normalize `pyformlang` FSTs
Canonical tokens and per-profile order

↓

3. Recognize Four DFAs
Acceptance, missing categories and traces

↓

4. Validate EBNF / `textX`
Candidate DSL → validated HTML

`src/pipeline.py` orchestrates the four stages. React and Streamlit present the same Python results; the browser does not reproduce the classification rules.

03 Extraction with regex

The extractor preserves source mentions, finds section boundaries and resolves overlapping matches. Contact spans are protected from accidental skill extraction.

Context matters: ambiguous names are restricted to skill sections. A longer mention such as `GitHub Actions` takes precedence over the overlapping `GitHub` match. Results remain in a structured data model and can be saved as JSON. Extraction records evidence; it does not decide profile acceptance.

04 Normalization with FSTs

The cascade **CLEAN** → **ALIAS**, plus an EXPERIENCE transducer, maps textual variants to canonical symbols. FILTER passes prepare a separate order for each profile.

$$T = (Q, \Sigma, \Gamma, \delta, \omega, q_0, F)$$

Here Σ and Γ are input and output alphabets; δ specifies transitions and ω their outputs. The transducers execute in `pyformlang`.

`JS` ↦ `JAVASCRIPT` `React.js` ↦ `REACT`
`NodeJS` ↦ `NODE_JS` `Postgres` ↦ `POSTGRESQL`

Exact years remain metadata; positive experience is abstracted as the finite symbol `EXPERIENCE_PRESENT`. Unknown skills remain visible.

05 Recognition with four DFAs

Each profile has a deterministic, total automaton with an explicit trap state:

$$M_i = (Q_i, \Sigma_i, \delta_i, q_0, F_i), \qquad \delta_i : Q_i \times \Sigma_i \to Q_i.$$

Acceptance means membership in the profile language:

$$w_i \in L(M_i) \iff \delta_i^*(q_0, w_i) \in F_i.$$

The prepared sequence w_i contains relevant skills in that profile's category order. A candidate may satisfy several profiles.

Machine Learning: accepted trace



One accepted path, with shortened state labels. Category loops and trap transitions are omitted here; complete models are in the repository.

Pattern decisions

Full Stack requires frontend, backend, database and version-control evidence, with defined alternatives and optional categories. The other three profiles require every category in their respective patterns.

Strict token mode preserves order: an out-of-order or foreign symbol reaches the trap state. Résumé mode prepares each profile's input first.

06 A validated candidate language

The DSL represents personal data, contacts, experience, education, skills and accepted profiles. Repeated records are supported.

```
candidate {
  personal { name "Wednesday Addams"; }
  contact { }
  experience { total_years 3; }
  education { }
  skills { skill JAVASCRIPT; ... }
  classification { accepted full_stack; }
}
```

Abbreviated illustration; the ellipsis is not valid DSL syntax.

`textX` parses the document and domain checks enforce valid contacts, vocabulary and uniqueness. **HTML is generated only after validation**; values are escaped. Its runtime parser uses PEG semantics, while the language structure is documented in EBNF.

07 Results and evidence

883

automated tests passing

Verified 11 October 2026: extraction boundaries, alias transduction, canonical order, DFA acceptance and rejection, DSL errors, HTML generation and API integration. The React production build also compiles.

End-to-end example

`JS, React.js, NodeJS, Postgres, Git`
⇓ extraction and FST normalization
`JAVASCRIPT, REACT, NODE_JS,`
`POSTGRESQL, GIT`
⇓ Full Stack DFA accepts
Validated candidate DSL ⟶ HTML profile.

08 Limits and next steps

Rule coverage limits extraction; negation and skill proficiency are not modeled. Employers, institutions and periods are not reliably reconstructed. Grammar validity does not establish factual truth, and acceptance does not rank candidates.

Next evaluation: an annotated résumé corpus to measure extraction precision and recall. Passing tests are evidence of programmed behavior, not recruitment accuracy.

Selected references

[1] J. Romero. *Pyformlang: An Educational Library for Formal Language Manipulation.* SIGCSE, 2021. doi:10.1145/3408877.3432464.

[2] I. Dejanović et al. *TextX: A Python tool for Domain-Specific Languages implementation.* Knowledge-Based Systems 115, 2017. doi:10.1016/j.knosys.2016.10.023.

[3] Python Software Foundation, *re documentation*; textX contributors, *Grammar and Meta-model documentation.* Full review: `docs/literature_review.md`.